

Day 7: Infrastructure as Code (IaC) with Terraform

1. Introduction to Infrastructure as Code (IaC)

Definition

Infrastructure as Code (IaC) is the practice of provisioning and managing infrastructure using code instead of manual processes.

Benefits

- Automation of infrastructure provisioning
 - Consistency across environments
 - Version control and rollback
 - Faster deployments
-

Scenario

Manual infrastructure setup

Problem:

- Errors due to manual steps
- Inconsistent environments

Solution:

- Use IaC with Terraform

Result:

- Repeatable and reliable infrastructure
-

Case Study

Enterprise DevOps Team

- Migrated manual setup to Terraform

Result:

- Reduced provisioning time from days to minutes
- Improved consistency

2. GCP Provider for Terraform

Definition

Provider is a plugin that allows Terraform to interact with cloud platforms like GCP.

Example

```
provider "google" {  
  project = "my-project-id"  
  region  = "asia-south1"  
}
```

Authentication

- Using service account key
 - Using gcloud authentication
-

Scenario

Need to provision GCP resources

Solution:

- Configure GCP provider

Result:

- Terraform can create and manage resources
-

Case Study

Cloud Migration Project

- Used Terraform GCP provider

Result:

- Automated infrastructure provisioning
-

3. Writing Reusable Modules

Definition

Modules are reusable Terraform configurations that help standardize infrastructure.

Example Structure

modules/

 vpc/

 gke/

 iam/

Example Module

```
module "vpc" {  
  source = "../modules/vpc"  
  network_name = "my-network"  
}
```

Benefits

- Reusability
 - Standardization
 - Easier maintenance
-

Scenario

Multiple environments

Problem:

- Duplicate code

Solution:

- Use modules

Result:

- Cleaner and reusable code
-

Case Study

Large Organization

- Created modules for VPC, GKE, IAM

Result:

- Faster infrastructure setup
 - Standardized architecture
-

4. State Management, Backend Configs, Workspaces

Terraform State

Definition

State file stores current infrastructure details

Backend Configuration

Definition

Remote storage for state (recommended)

Example:

```
terraform {  
  backend "gcs" {  
    bucket = "my-tf-state-bucket"  
    prefix = "terraform/state"  
  }  
}
```

Workspaces

Definition

Separate environments using same configuration

Example:

- dev
 - test
 - prod
-

Scenario

Multiple teams working

Problem:

- State conflicts

Solution:

- Use remote backend + workspaces

Result:

- Safe and isolated environments
-

Case Study

Enterprise Deployment

- Uses GCS backend

Result:

- Centralized and secure state management
-

5. Resource Dependency and Lifecycle Management

Dependency Management

Definition

Terraform automatically determines resource creation order

Explicit Dependency

`depends_on = [google_compute_network.vpc]`

Lifecycle Rules

Example:

```
lifecycle {  
  prevent_destroy = true  
  create_before_destroy = true  
}
```

Scenario

Database accidentally deleted

Problem:

- Data loss

Solution:

- Use lifecycle rules

Result:

- Protected critical resources
-

Case Study

Production Environment

- Enabled prevent_destroy for DB

Result:

- Avoided accidental deletion
-

6. Hands-On: Provision GKE + VPC + IAM + Registry

Step 1: Initialize Terraform

```
terraform init
```

Step 2: Define Provider

```
provider "google" {
```

```
project = "my-project-id"
region = "asia-south1"
}
```

Step 3: Create VPC

```
resource "google_compute_network" "vpc" {
  name = "my-vpc"
  auto_create_subnetworks = false
}
```

Step 4: Create Subnet

```
resource "google_compute_subnetwork" "subnet" {
  name          = "my-subnet"
  ip_cidr_range = "10.0.0.0/24"
  region       = "asia-south1"
  network      = google_compute_network.vpc.id
}
```

Step 5: Create GKE Cluster

```
resource "google_container_cluster" "primary" {
  name     = "my-gke-cluster"
  location = "asia-south1-a"

  network    = google_compute_network.vpc.name
  subnetwork = google_compute_subnetwork.subnet.name

  initial_node_count = 1
}
```

Step 6: Create IAM Role Binding

```
resource "google_project_iam_member" "viewer" {  
  role = "roles/viewer"  
  member = "user:example@gmail.com"  
}
```

Step 7: Create Artifact Registry

```
resource "google_artifact_registry_repository" "repo" {  
  location = "asia-south1"  
  repository_id = "my-repo"  
  format = "DOCKER"  
}
```

Step 8: Plan and Apply

terraform plan

terraform apply

End-to-End Case Study (Day 7 Summary)

Scenario: Automated Infrastructure Setup

Problem:

- Manual infrastructure setup
 - Time-consuming and error-prone
-

Solution

1. Use Terraform for IaC
2. Configure GCP provider
3. Create reusable modules
4. Manage state remotely
5. Define dependencies and lifecycle rules

Result

- Automated provisioning
- Consistent environments
- Faster deployment
- Reduced human errors